GoParking

Parking Allotment Management System

Project Type	B.Tech CSE Minor Project
Primary Language	Java (Core Java + DSA)
Interface	Streamlit web application
Data Storage	SQLite database

1. Introduction

GoParking is a web-based Parking Allotment Management System designed to simplify vehicle entry, slot allocation, vehicle exit, fee calculation, and parking record management. The project combines a Java backend that applies Data Structures and Algorithms (DSA) with a Streamlit interface and an SQLite database for persistent parking history and active-session records.

2. Objectives

- Automate vehicle parking and removal.
- Allocate the nearest available slot using an ordered set of available slot numbers.
- Find parked vehicles quickly by registration number and search vehicles by color.
- Prevent duplicate registration entries and reject vehicles when all slots are occupied.
- Calculate parking fees based on vehicle type and duration.
- Preserve parking history and payment records in SQLite and export history as CSV.

3. Technology Stack

Component	Technology / Role
Backend	Core Java; socket server for requests from Streamlit
Frontend	Python + Streamlit for dashboard and forms
Data structures	TreeSet, HashMap, ArrayList
Persistence	SQLite tables for parking events and payments
Export	CSV download for parking history

4. DSA Concepts Used

Data Structure	Purpose in GoParking	Typical complexity
TreeSet	Maintains free slot numbers in sorted order; first available slot can be selected.	Add/remove: O(log n); first element access: O(log n) in Java TreeSet API
HashMap	Maps registration numbers to parking slots for fast lookup and duplicate checking.	Average lookup/insert/remove: O(1)
ArrayList	Stores vehicle and parking history lists in memory.	Append: amortized O(1); search: O(n)

5. Main Modules

- **Dashboard:** Displays total, occupied, and available slots, plus a visual slot map.
- **Park Vehicle:** Validates input, blocks duplicate registrations, and assigns an available slot.
- **Remove Vehicle:** Records exit time, calculates the fee, frees the slot, and records the fee.
- **Search:** Finds an active vehicle by registration number or lists active vehicles by color.
- **Parking Slots:** Shows each slot's current availability and vehicle details.
- **Parking History:** Displays previous parking sessions and supports CSV export.
- **Payments:** Shows fee records stored in the database; it does not by itself prove a real online payment gateway is connected.
- **Fee Calculator:** Estimates cost using the configured rates: Car ■20/hour and Bike ■10/hour.

6. System Workflow

1. The user submits a parking or search request through the Streamlit interface.
2. Streamlit sends a command to the Java server over a local TCP socket.
3. Java validates the request and uses TreeSet/HashMap/ArrayList to manage the in-memory parking state.
4. Streamlit records parking events and fee records in SQLite.
5. On a Java server restart, active vehicles can be restored from SQLite into Java using the RESTORE command.

7. Testing and Results

Test	Observed result
Park and remove vehicle	Successful; history and exit fee were recorded.
Restart Java server	Active vehicle appeared again after restoration.
Search by registration	Removed vehicle was reported as not found among active vehicles.
Search by color	No active red vehicle was found after removal.
CSV export	Parking history CSV opened in Microsoft Excel.
Payments page	Fee records and total recorded fees were displayed.
Duplicate registration	System returned 'Vehicle already parked'.
Full parking lot	System returned 'Parking lot is full' when capacity was reached.
Fee calculator	Car/Bike hourly fee calculation was checked.

8. Limitations and Future Scope

- Current Java parking state is in memory and is restored from SQLite by the Streamlit app after a server restart.
- The demo login is not a production-grade authentication system.
- Payments are recorded as database entries; a real payment gateway is not integrated.
- Future enhancements may include secure user roles, a real payment gateway, configurable slot capacity, vehicle categories, reports, and deployment to a hosted server.

9. Conclusion

GoParking demonstrates how DSA concepts can solve practical parking-allocation problems. TreeSet helps select an available slot in sorted order, HashMap supports fast vehicle lookup and duplicate prevention, and ArrayList stores in-memory lists. The Streamlit interface makes the functions accessible, while SQLite preserves parking events and recorded fees. The completed tests indicate that the main demo workflows behave as expected.